

The complete Paradox map creation pipeline

Every Paradox grand strategy map is built from the same fundamental architecture: layered bitmaps mapped to structured data files, processed by either the legacy Clausewitz engine (EU4, HOI4) or the modern Jomini framework (CK3, Vic3). This document provides the exhaustive technical specification needed to build a functional map from scratch for any of these four games. The pipeline has remained remarkably consistent across two decades of engine evolution, with Jomini introducing visual tooling and format modernization (PNG over BMP, 16-bit heightmaps, integrated editors) while preserving the core paradigm of RGB-color-coded province bitmaps linked to semicolon-delimited definition files.

Phase 1: The engine architecture

Clausewitz — the 32-bit foundation powering EU4 and HOI4

The Clausewitz engine debuted with Europa Universalis III ([IndieDB](#)) in 2007 ([Wikipedia](#)) and has powered every Paradox grand strategy title since. ([IGDB](#)) Written in C++, ([NamuWiki](#)) it ([Wikipedia](#)) runs as a **32-bit DirectX 9 application** with limited multi-threading. The engine's map system composites multiple bitmap layers into a real-time 3D terrain view through a fixed pipeline:

1. **Heightmap mesh generation** — The engine reads (`heightmap.bmp`) (8-bit greyscale) and builds a 3D terrain mesh. ([Translate](#)) Pixel brightness maps directly to elevation, with **sea level fixed at greyscale value 94-95**. ([Strategicclub](#)) Everything below renders as water. ([Strategicclub](#)) ([Paradox Wikis](#))
2. **Normal map lighting** — (`world_normal.bmp`) (24-bit RGB, exactly half the map resolution) provides per-pixel surface normals for lighting calculations. ([Modding Tutorials](#)) ([Strategicclub](#)) Red = X-axis direction, Green = Y-axis, Blue = Z-axis perpendicularity. This creates the illusion of micro-detail on the terrain surface.
3. **Terrain texture application** — (`terrain.bmp`) (8-bit indexed) assigns terrain texture tiles from atlas DDS files. Each pixel's palette index maps to a terrain category defined in (`terrain.txt`), ([Modding Tutorials](#)) ([PDX Tools](#)) which in turn references a specific tile position in (`atlas0.dds`).
4. **Seasonal colormap overlay** — DDS files (`colormap_summer.dds`, etc.) at half resolution are blended onto the terrain surface, providing the characteristic "satellite photo" look. ([Modding Tutorials](#)) Season dates are controlled by (`seasons.txt`).
5. **Province border detection** — The engine scans (`provinces.bmp`) (24-bit RGB) pixel-by-pixel, building a lookup table of RGB → Province ID using (`definition.csv`).

(Modding Tutorials +2) Borders are detected wherever adjacent pixels differ in color. **This is the single most critical file** — a single anti-aliased pixel creates an undefined color that crashes or corrupts the map.

6. **River rendering** — (`rivers.bmp`) (8-bit indexed) overlays river geometry (Modding Tutorials) using palette-specific colors for source points, flow direction, tributaries, and width grades.
7. **Tree model placement** — (`trees.bmp`) (8-bit indexed, approximately 1/7th resolution) determines where 3D tree models are instanced on the terrain. (Modding Tutorials) (PDX Tools)

The Clausewitz engine provides **no built-in map editing tools**. As engine team lead John Wordsworth stated at PDXCON 2019: "Modders always ask us why we don't give them the tools we use to make the games, but we don't have any. People are the tools." (VentureBeat) The only visual aid is the **Nudge tool**, accessed via the (`-nudge`) launch parameter in EU4 (Modding Tutorials) or (`-debug`) in HOI4, which allows repositioning of city/unit/port markers but cannot edit terrain or provinces.

Jomini — the 64-bit framework behind CK3 and Vic3

Introduced with Imperator: Rome in 2019 and named after Swiss general Antoine-Henri Jomini, this framework (Wikipedia) sits as a mid-layer between individual game projects and the underlying Clausewitz codebase. It is specifically designed for (Wikipedia) "top-down, map-based games" and represents a complete rewrite of the graphics pipeline.

Key architectural changes from Clausewitz:

- **64-bit native** with DirectX 11 rendering (Wikipedia) and full multi-threading support (NamuWiki)
- **PNG replaces BMP** for all map images (`provinces.png`), (`heightmap.png`), (`rivers.png`) (Paradox Wikis)
- **16-bit greyscale heightmaps** (Paradox Plaza) (65,536 elevation levels vs. Clausewitz's 256), enabling dramatically more detailed terrain (Paradox Wikis)
- **Integrated map editor** accessible via (`-mapeditor`) launch option (Paradox Plaza) (Paradox Wikis) or (`map_editor`) console command (Paradox Wikis) — supports heightmap sculpting, terrain painting, object placement, and province-to-state assignment (Paradox Plaza)
- **Terrain mask system** — instead of a single indexed (`terrain.bmp`), CK3 uses **63+ individual mask PNG files** at full map resolution in (`gfx/map/terrain/`), (Paradox Wikis) each controlling the opacity of one terrain material
- **Separate locator files** replace the monolithic (`positions.txt`) — (`building_locators.txt`), (`combat_locators.txt`), (`player_stack_locators.txt`), and (`siege_locators.txt`) in (`gfx/map/map_object_data/`) (Paradox Wikis)

- **Auto-generated heightmap derivatives** — `packed_heightmap.png` and `indirection_heightmap.png` (1/32 resolution) are created by the map editor's "repack" operation, [Paradox Wikis](#) optimizing terrain rendering [Paradox Wikis](#) [Paradox Wikis](#)
- **Map data folder renamed** from `map/` to `map_data/`

The Jomini engine still uses the same fundamental RGB-province paradigm, but Vic3 takes a radical step further by **eliminating** `definition.csv` **entirely** — province IDs are the hex color codes themselves, referenced as `xHHHHHH` throughout all game files. [Steam Community](#)

[Paradox Plaza](#)

Comparative architecture summary

Feature	Clausewitz (EU4/HOI4)	Jomini (CK3/Vic3)
Bit architecture	32-bit	64-bit
Graphics API	DirectX 9	DirectX 11
Province map format	<code>provinces.bmp</code> (24-bit RGB BMP)	<code>provinces.png</code> (24-bit RGB PNG)
Heightmap format	<code>heightmap.bmp</code> (8-bit greyscale)	<code>heightmap.png</code> (16-bit greyscale)
Map folder	<code>map/</code>	<code>map_data/</code>
Province definition	<code>definition.csv</code> (ID;R;G;B;Name)	CK3: <code>definition.csv</code> ; Vic3: none (hex = ID)
Terrain assignment	<code>terrain.bmp</code> (indexed) + <code>terrain.txt</code>	CK3: <code>common/province_terrain/</code> ; Vic3: <code>province_terrains.txt</code>
Positions	<code>positions.txt</code> (single file)	Separate locator <code>.txt</code> files
Built-in editor	None (Nudge tool only)	Full map editor with terrain painting
River format	<code>rivers.bmp</code> (8-bit indexed BMP)	<code>rivers.png</code> (8-bit indexed PNG)
Colormap system	Seasonal DDS files (spring/summer/autumn/winter)	<code>colormap.dds</code> + terrain masks

Phase 2: The file manifest

EU4 complete file manifest (/map/ folder)

Every file listed below is required to boot a functional EU4 map unless marked optional.

File	Format	Dimensions	Bit Depth	Purpose
provinces.bmp	BMP	$W \times H$ (vanilla: 5632×2048)	24-bit RGB	Province color map
definition.csv	CSV (⌈;⌋ delimited, ANSI)	—	Text	Province ID ↔ RGB mapping
default.map	Clausewitz script	—	Text	Master config: dimensions, file refs, sea/lake lists
heightmap.bmp	BMP	$W \times H$	8-bit greyscale	Elevation data
world_normal.bmp	BMP	$W/2 \times H/2$	24-bit RGB	Bump/normal map for lighting
terrain.bmp	BMP	$W \times H$	8-bit indexed	Terrain type assignment
terrain.txt	Clausewitz script	—	Text	Terrain category definitions
rivers.bmp	BMP	$W \times H$	8-bit indexed	River paths and flow
trees.bmp	BMP	$\sim W/8 \times \sim H/7$	8-bit indexed	Forest placement
positions.txt	Clausewitz script	—	Text	City/unit/port/text positions
adjacencies.csv	CSV (⌈;⌋ delimited)	—	Text	Straits, canals, special connections
continent.txt	Clausewitz script	—	Text	Province → continent assignments
climate.txt	Clausewitz script	—	Text	Province climate/winter

File	Format	Dimensions	Bit Depth	Purpose
				severity
area.txt	Clausewitz script	—	Text	Province → area groupings
region.txt	Clausewitz script	—	Text	Area → region groupings
superregion.txt	Clausewitz script	—	Text	Region → superregion groupings
seasons.txt	Clausewitz script	—	Text	Season date ranges and color variance
trade_winds.txt	Clausewitz script	—	Text	Trade wind province assignments
provincegroup.txt	Clausewitz script	—	Text	Code-only province groups (optional)
ambient_object.txt	Clausewitz script	—	Text	Ambient 3D models (seagulls, canals)
terrain/colormap_summer.dds	DDS (ARGB 8.8.8.8, no mipmaps)	$W/2 \times H/2$	32bpp	Summer color overlay
terrain/colormap_winter.dds	DDS	$W/2 \times H/2$	32bpp	Winter color overlay
terrain/colormap_spring.dds	DDS	$W/2 \times H/2$	32bpp	Spring color overlay
terrain/colormap_autumn.dds	DDS	$W/2 \times H/2$	32bpp	Autumn color overlay
terrain/colormap_water.dds	DDS	$W/2 \times H/2$	32bpp	Underwater color overlay

Dimension constraints: Width and height must be **multiples of 128** (Strategicclub) (some sources specify 256). Vanilla is 5632×2048 . (Strategicclub) The max_provinces value in default.map must exceed the highest province ID.

HOI4 complete file manifest (`/map/` folder)

HOI4 shares most EU4 files but adds supply/logistics infrastructure and uses a richer `definition.csv`.

File	Format	Dimensions	Bit Depth	Purpose
provinces.bmp	BMP	$W \times H$ (vanilla: 5632 × 2048)	24-bit RGB	Provinces
definition.csv	CSV (⌈;⌋ delimited)	—	Text	Extended ID;R;G
default.map	Clausewitz script	—	Text	Master script
heightmap.bmp	BMP	$W \times H$	8-bit greyscale	Elevation
world_normal.bmp	BMP	$W \times H$	24-bit RGB	Normal map
terrain.bmp	BMP	$W \times H$	8-bit indexed	Visual terrain
rivers.bmp	BMP	$W \times H$	8-bit indexed	River placement
trees.bmp	BMP	$W \times H$	8-bit indexed	Forest placement
adjacencies.csv	CSV (⌈;⌋ delimited)	—	Text	Straits and boundaries
adjacency_rules.txt	Clausewitz script	—	Text	Conditional rules
continent.txt	Clausewitz script	—	Text	Continent definitions
seasons.txt	Clausewitz script	—	Text	Seasonal effects
ambient_object.txt	Clausewitz script	—	Text	Ambient object definitions

File	Format	Dimensions	Bit Depth	Purpose
<code>positions.txt</code>	Clausewitz script	—	Text	Province
<code>unitstacks.txt</code>	Clausewitz script	—	Text	Unit m
<code>supply_nodes.txt</code>	Text	—	Text	Supply
<code>railways.txt</code>	Text	—	Text	Railwa
<code>rocketsites.txt</code>	Text	—	Text	Rocket
<code>weatherpositions.txt</code>	Text	—	Text	Weath
<code>strategicregions/*.txt</code>	Clausewitz script	—	Text	Strateg
<code>terrain/colormap_rgb_cityemissivemask_a.dds</code>	DDS (DXT5, mipmaps)	$W/2 \times H/2$	32bpp	RGB =
<code>terrain/colormap_water_0.dds</code>	DDS (DXT5)	$W/2 \times H/2$	32bpp	Water c
<code>terrain/colormap_water_1.dds</code>	DDS (DXT5)	$W/4 \times H/4$	32bpp	Water c
<code>terrain/colormap_water_2.dds</code>	DDS (DXT5)	$W/8 \times H/8$	32bpp	Water c
<code>terrain/atlas0.dds</code>	DDS (DXT5)	—	—	Terrair

HOI4-specific constraints: Both width and height must be multiples of **256**. Total pixel area cannot exceed approximately **13,107,200 pixels**. Minimum province size is **8 pixels**.

`(Steam Community)` `(Agile Health)` Province bounding box cannot exceed 1/8th of total map width or height. Vanilla has ~13,252 provinces with a practical ceiling of ~19,000–22,000. `(Steam Community)`

CK3 complete file manifest (`/map_data/` and `/gfx/map/`)

File	Format	Dimensions	Bit Depth
<code>map_data/provinces.png</code>	PNG	$W \times H$ (vanilla: 8192×4096)	24-bit RGB (no alpha)
<code>map_data/definition.csv</code>	CSV (<code>(;)</code> delimited)	—	Text
<code>map_data/default.map</code>	Jomini script	—	Text
<code>map_data/heightmap.png</code>	PNG	$W \times H$	16-bit greyscale
<code>map_data/heightmap.heightmap</code>	Text	—	Text
<code>map_data/packed_heightmap.png</code>	PNG	Auto- generated	Auto
<code>map_data/indirection_heightmap.png</code>	PNG	$W/32 \times H/32$	Auto
<code>map_data/rivers.png</code>	PNG	$W \times H$	8-bit indexed
<code>map_data/adjacencies.csv</code>	CSV (<code>(;)</code> delimited)	—	Text
<code>gfx/map/terrain/flatmap.dds</code>	DDS	$W \times H$	—
<code>gfx/map/terrain/colormap.dds</code>	DDS	$W/4 \times H/4$	—
<code>gfx/map/terrain/detail_intensity.tga</code>	TGA	$W \times H$	—
<code>gfx/map/terrain/detail_index.tga</code>	TGA	$W \times H$	—
<code>gfx/map/terrain/*_mask.png</code> (×63)	PNG	$W \times H$	—
<code>gfx/map/water/flowmap.dds</code>	DDS	$W/4 \times H/4$	—

File	Format	Dimensions	Bit Depth
<code>gfx/map/water/foam_map.dds</code>	DDS	W/8 × H/8	—
<code>gfx/map/water/watercolor_rgb_waterspec_a.dds</code>	DDS	W/2 × H/2	—
<code>gfx/map/surround_map/surround_mask.dds</code>	DDS	W/2 × H/2	—
<code>gfx/map/surround_map/surround_fade.dds</code>	DDS	W/8 × H/8	—
<code>gfx/map/map_object_data/building_locators.txt</code>	Text	—	—
<code>gfx/map/map_object_data/combat_locators.txt</code>	Text	—	—
<code>gfx/map/map_object_data/player_stack_locators.txt</code>	Text	—	—
<code>gfx/map/map_object_data/siege_locators.txt</code>	Text	—	—
<code>content_source/map_objects/masks/*_mask</code> (×21+)	PNG	W/2 × H/2	—
<code>common/province_terrain/00_province_terrain.txt</code>	Text	—	—
<code>common/landed_titles/00_landed_titles.txt</code>	Text	—	—

CK3-specific notes: The province map defines **baronies** (the smallest map unit), not counties.

[Paradox Wikis](#) Counties group 2-5 baronies [Paradox Interactive](#) and are defined in `landed_titles`. [GitHub](#) Vanilla has approximately **7,900 provinces**. [Steam Community](#) The `provinces.png` must NOT have an alpha channel [Paradox Plaza](#) — RGBA causes an immediate CTD. After any heightmap edit, you must **repack** via the map editor to regenerate `packed_heightmap.png` and `indirection_heightmap.png`. [Paradox Wikis +2](#)

Vic3 complete file manifest (`/map_data/` and related)

File	Format	Dimensions	Bit Depth	Purpose
<code>map_data/provinces.png</code>	PNG	W × H (vanilla: ~8192 × 3616)	24-bit RGB	Province map (hex color = province ID)

File	Format	Dimensions	Bit Depth	Purpose
<code>map_data/default.map</code>	Jomini script	—	Text	Sea province list (<code>sea_starts</code>)
<code>map_data/heightmap.png</code>	PNG	$W \times H$	16-bit greyscale	Elevation data
<code>map_data/packed_heightmap.png</code>	PNG	Auto-generated	Auto	Engine-optimized heightmap
<code>map_data/indirection_heightmap.png</code>	PNG	$W/32 \times H/32$	Auto	Tile indirection
<code>map_data/rivers.png</code>	PNG	$W \times H$	8-bit indexed	River paths
<code>map_data/adjacencies.csv</code>	CSV	—	Text	Special connections
<code>map_data/province_terrains.txt</code>	Text	—	Text	Auto-generated terrain assignments
<code>map_data/state_regions/*.txt</code>	Jomini script	—	Text	State region definitions with hubs
<code>common/strategic_regions/*.txt</code>	Jomini script	—	Text	Strategic region groupings
<code>common/terrain/01.terrain.txt</code>	Text	—	Text	Terrain type definitions

Vic3-specific notes: There is no `definition.csv`. The hex color of each province IS its identifier, referenced as `xHHHHHH` (e.g., `x4E7B2A`) in all game files. [Steam Community](#) This is a fundamental departure from every other Paradox game. Every land state requires a minimum of **4 provinces** (one per hub type: city, farm, mine, wood), or **5 provinces** for coastal states (adding port). [Steam Community](#) A minimum of **21 provinces** is needed for the game to boot.

[Steam Community](#)

Phase 3: The graphical pipeline

Step 1 — The heightmap (foundation layer)

The heightmap defines the 3D terrain mesh and must be created first, as all other visual layers depend on it.

For Clausewitz games (EU4/HOI4): Create an 8-bit greyscale BMP at the target map resolution. Black (0) = deepest ocean floor; white (255) = highest peak. **Sea level is at greyscale value 94–95** — any pixel below this renders as water. (Paradox Wikis) Transitions between elevations must be **smooth and gradual**; (Strategicclub) abrupt jumps (e.g., 90 → 110 without intermediate values) cause Z-collision glitches and jagged visual artifacts. (Modding Tutorials) When saving from GIMP, check "Do not write colourspace information" to prevent engine parsing errors. (Paradox Wikis)

For Jomini games (CK3/Vic3): Create a 16-bit greyscale PNG at target resolution (vanilla: 8192 × 4096 for CK3). (Paradox Plaza) The 16-bit depth provides **65,536 elevation levels** versus Clausewitz's 256. (Paradox Wikis) The water level is controlled by the `WATERLEVEL` define in `common/defines/graphic/00_graphics.txt`. (Paradox Wikis) After creating or modifying the heightmap, you must import it into the map editor (`-mapeditor`) and run a **repack** operation, which generates `packed_heightmap.png` and `indirection_heightmap.png`. (Paradox Wikis) The `heightmap.heightmap` metadata file must contain `original_heightmap_size` matching your PNG dimensions. (Kirusa) (Paradox Wikis)

Real-world data sources: NASA Blue Marble heightmap data, (Paradox Wikis) USGS Earth Explorer DEM files, or procedural generation via World Machine, Gaea, or FastNoiseLite (used by the RandomParadox tool). (GitHub)

Step 2 — The province map (the critical layer)

This is the single most error-sensitive file in the entire pipeline. Every pixel must be exactly one solid RGB color with **zero tolerance for anti-aliasing, transparency, or color blending**.

Strict rules for all games:

- **Anti-aliasing must be completely disabled.** Use the Pencil tool (never Brush), set Paint Bucket tolerance to 0 with anti-alias off, (Paradox Wikis) and use **Nearest Neighbor** interpolation for any resizing operation.
- **RGB (0, 0, 0) — pure black — is forbidden** in EU4 and will crash the game. HOI4 reserves it for province 0.
- Every province must be a **contiguous region** of a single unique RGB color.
- **No duplicate RGB values.** Two provinces sharing a color will silently merge.
- In HOI4, **no X-crossings**: four different province colors may not meet at a single pixel corner. The engine logs: `"Map invalid X crossing. Please fix pixels at coords"`.

[GitHub](#)

- HOI4 enforces a minimum province size of **8 pixels** [Agile Health](#) and a maximum bounding box of 1/8th of total map dimensions.

Format specifics by game:

Game	File	Format	Bit Depth
EU4	provinces.bmp	Windows BMP	24-bit RGB
HOI4	provinces.bmp	Windows BMP	24-bit RGB (32-bit causes CTD) Translate
CK3	provinces.png	PNG	8-bit or 24-bit RGB (NO alpha channel)
Vic3	provinces.png	PNG	24-bit RGB

Color generation: Use the Clausewitz-Color-Factory tool to generate 1-50,000 unique province colors with configurable minimum contrast and brightness sorting. [GitHub](#) Alternatively, RandomParadox generates complete province maps procedurally. [GitHub](#)

Step 3 — The terrain map

Clausewitz (EU4/HOI4): Create [terrain.bmp](#) as an **8-bit indexed color BMP** at full map resolution. The file uses a color palette (color table) where each pixel stores a single byte indexing into the palette. These indices map to terrain categories via [terrain.txt](#):

```
terrain = {
  grasslands = {
    type = plains
    color = { 0 4 }    # palette indices 0 and 4 both count as grasslands
  }
  forest = {
    type = forest
    color = { 1 }
  }
}
```

The engine automatically assigns terrain to each province by counting which terrain-type pixels are most prevalent within that province's borders. Ties break in favor of the **lowest color index**.

[PDX Tools](#) The [terrain_override](#) directive in [terrain.txt](#) can manually force any province to a specific terrain. [PDX Tools](#)

Critical: Only Photoshop and GIMP properly preserve the indexed color table. MS Paint, Paint.NET, and LibreOffice all regenerate the palette unpredictably, **silently corrupting the file**.

Paradox Wikis

Jomini (CK3): Uses 63+ individual `_mask.png` files in `gfx/map/terrain/`, each at full map resolution, painted via the integrated map editor's terrain brush. Province terrain types are set explicitly in `common/province_terrain/00_province_terrain.txt` (e.g., `1 = plains`, `2 = farmlands`). Paradox Wikis

Jomini (Vic3): Terrain is auto-generated via the `terrain.regenerate` console command, which reads geography trigger scripts and outputs `province_terrains.txt` mapping each hex province ID to a terrain type. Steam Community

Step 4 — The rivers map

Rivers use the same indexed-color paradigm across all four games, with specific palette colors encoding flow semantics.

Universal river color codes:

Color	RGB Value	Purpose
Green	<code>(0, 255, 0)</code>	River source — the origin point of a river system
Red	<code>(255, 0, 0)</code>	Flow-in tributary — marks where a branch joins the main channel
Yellow	<code>(255, 252, 0)</code> in EU4; <code>(255, 255, 0)</code> in HOI4	Flow-out split — river branches outward
Light blue	<code>(0, 225, 255)</code>	Narrowest river width
Blue	<code>(0, 200, 255)</code>	Narrow river
Medium blue	<code>(0, 150, 255)</code>	Medium river
Dark blue	<code>(0, 100, 255)</code>	Wide river
Darkest blue	<code>(0, 0, 200)</code>	Widest river

Strict drawing rules:

- Rivers must be **exactly 1 pixel wide**. [Agile Health](#) Two-pixel-wide rivers fail to render. [Paradox Wikis](#)
- River pixels must be **orthogonally adjacent** (up/down/left/right) — diagonal connections fail. [Paradox Wikis](#)
- Each contiguous river system may have **exactly one green source pixel**. [Paradox Wikis](#)
- The indexed color palette from the vanilla file must be preserved exactly. [Paradox Wikis](#) In GIMP, open with "keep color profile" and export via "overwrite" to maintain indexing.
- In Vic3, river width is controlled by 13 gradient colors spanning from [TH_MIN](#) (default 1.25) to [TH_MAX](#) (default 2.75), defined in [/common/defines/jomini/rivers.txt](#). [Paradox Wikis](#) [Paradox Wikis](#)

Known GIMP bug (HOI4): GIMP sets the DIB header to declare 256 palette colors, but HOI4 expects 0. This produces ["MAP_ERROR: Palette in rivers.bmp is probably not correct"](#). Fix by hex-editing addresses [0x2F](#) and [0x33](#) from [01](#) to [00](#). [Translate](#)

Step 5 — The normal map and colormaps

Normal map (EU4/HOI4): Generate [world_normal.bmp](#) from the heightmap using NVIDIA Texture Tools or Photoshop CC's Filter → 3D → Generate Normal Map. [Strategicclub](#) **In EU4, this must be exactly half the map resolution** (e.g., 2816 × 1024 for vanilla). Wrong dimensions cause a distinctive **neon glow effect** on the entire world. [wordpress](#) In HOI4, the normal map is at full resolution. RGB channels encode surface normal vectors: Red = X direction (0 = west, 255 = east), Green = Y direction (0 = south, 255 = north), Blue = Z perpendicularity (128 = flat, 255 = facing camera). [Translate](#) When generating in GIMP, **invert the Y axis**. [GitHub](#) [Translate](#)

Colormaps (EU4): Create four seasonal DDS files ([colormap_spring.dds](#) through [colormap_winter.dds](#)) plus [colormap_water.dds](#) in [/map/terrain/](#). Format: **ARGB 8.8.8.8, 32bpp, no mipmaps**. [Strategicclub](#) Resolution: exactly half map dimensions. These provide the satellite-imagery color tinting that gives the map its visual character.

Colormaps (HOI4): Uses [colormap_rgb_cityemissivemask_a.dds](#) where RGB channels provide map coloring and the **alpha channel doubles as a city night-light emissive mask** (higher opacity = brighter city lights). Water uses three LOD levels: [colormap_water_0.dds](#) (half resolution), [_1.dds](#) (quarter), [_2.dds](#) (eighth). All require **DXT5 compression with mipmaps**.

Colormaps (CK3/Vic3): Uses [gfx/map/terrain/colormap.dds](#) at 1/4 resolution, plus the water system files [watercolor_rgb_waterspec_a.dds](#) (1/2 resolution), [flowmap.dds](#) (1/4), and [foam_map.dds](#) (1/8). [Kirusa](#)

Step 6 — Trees

`trees.bmp` (EU4/HOI4) is an 8-bit indexed BMP (Modding Tutorials) at approximately **1/8th width** × **1/7th height** of the main map (e.g., ~704 × 293 for vanilla EU4). (PDX Tools) Specific palette indices declared in `default.map` via `tree = { 3 4 7 10 }` count as tree coverage. Heavy tree coverage can override terrain assignment — this is how provinces become jungle/forest even when `terrain.bmp` shows grasslands. (PDX Tools)

Phase 4: The data integration layer

definition.csv — the RGB ↔ ID bridge

This semicolon-delimited file is the critical link between the graphical province map and all gameplay data.

EU4 format:

```
province;red;green;blue;x;x           ← header row
1;130;0;48;Stockholm;x
2;104;0;41;Östergötland;x
```

Columns: `province_id;red;green;blue;name;x`. Encoding: **ANSI (Windows-1252)**.

HOI4 format (extended):

```
0;0;0;0;land;false;unknown;0         ← province 0 (no header)
1;86;46;34;land;false;forest;1
2;210;137;23;land;true;plains;1
3;40;23;137;sea;false;ocean;0
```

Columns: `ProvinceID;Red;Green;Blue;Type;Coastal;Terrain;Continent`. The `Type` field (`land`/`sea`/`lake`), `Coastal` flag (`true`/`false`), `Terrain` (gameplay terrain type), and `Continent` (integer) are HOI4-specific additions. HOI4 auto-generates a `definitions.csv.fixed.csv` on launch if the file is incomplete.

CK3 format:

```
0;0;0;0;;x;                           ← province 0 (unused)
1;42;3;128;Vestrland;x;
4;168;12;2;Austrland;x;
```

Columns: `ProvID;R;G;B;title;x;`. The "title" maps to barony-level titles (e.g., `b_vestrland`).

Vic3: No `definition.csv` exists. Province hex colors are used directly as identifiers with an `x` prefix: `x4E7B2A`.

default.map — the master configuration

This file declares map dimensions, references all other map files, and classifies provinces.

EU4 default.map structure:

```

width = 5632
height = 2048
max_provinces = 4348
sea_starts = { 1 2 3 ... }           # all sea province IDs
lakes = { 4100 4101 ... }            # lake province IDs
only_used_for_random = { 4019 ... }  # Random New World provinces
force_coastal = { }

definitions = "definition.csv"
provinces = "provinces.bmp"
positions = "positions.txt"
terrain = "terrain.bmp"
rivers = "rivers.bmp"
terrain_definition = "terrain.txt"
heightmap = "heightmap.bmp"
tree_definition = "trees.bmp"
continent = "continent.txt"
adjacencies = "adjacencies.csv"
climate = "climate.txt"
region = "region.txt"
superregion = "superregion.txt"
area = "area.txt"
provincegroup = "provincegroup.txt"
ambient_object = "ambient_object.txt"
seasons = "seasons.txt"
trade_winds = "trade_winds.txt"

tree = { 3 4 7 10 }                  # palette indices counted as trees

canal_definition = {
    name = "panama_canal"
    x = 1490
    y = 948
}

```

CK3 default.map uses a different structure — instead of width/height declarations, it defines province category ranges:

```

sea_zones = { 2134 2134 }           # range: start_ID end_ID
river_provinces = { ... }           # navigable river provinces
lakes = { ... }
ocean_region = { ... }              # interconnected sea zones
major_rivers = { ... }
impassable_mountains = { ... }      # wasteland provinces
impassable_seas = { ... }
externals = { ... }
max_provinces = 7901

```

Vic3 default.map is minimal, primarily containing `sea_starts` with hex province IDs.

adjacencies.csv — straits, canals, and special connections

Universal format (all games):

```

From;To;Type;Through;start_x;start_y;stop_x;stop_y;Comment
6;12;sea;1258;3008;1633;3000;1630;Skåne-Sjaelland
-1;-1;;-1;-1;-1;-1;-1;          ← MANDATORY terminator

```

Types by game:

- EU4: `sea` (strait), `canal`, `river`
- HOI4: `sea` (strait), `impassable`
- CK3: `sea`, `river_large`

The **terminator line** (`-1;-1;;-1;-1;-1;-1;-1;-1;`) is absolutely critical. Omitting it causes an **infinite loading screen** in CK3 or an infinite hang at startup in EU4/HOI4.

Coordinates can use `-1` to let the engine auto-calculate the visual crossing line position. The `Through` field specifies a province that must be controlled to use the crossing — if an enemy controls it, the crossing is blocked.

positions.txt — the EU4/HOI4 positioning system

Each province entry contains **7 coordinate pairs** for different map elements:

```

1={
  position={
    3173.000 1093.000    # 1. City graphic
    3168.000 1090.000    # 2. Unit (idle army/navy)
    3174.000 1095.000    # 3. Province name text
    3180.000 1085.000    # 4. Port model
    3170.000 1091.000    # 5. Trade route
    3175.000 1088.000    # 6. Fighting unit (combat)
    3172.000 1092.000    # 7. Trade wind icon
  }
  rotation={
    0.000 0.000 0.000 0.000 0.000 0.000 0.000
  }
  height={
    0.000 0.000 1.000 0.000 0.000 0.000 0.000
  }
}

```

Coordinate system: X maps 1:1 to bitmap pixel X. **Y is inverted:** $Y_positions = map_height - Y_image_editor$. Rotation uses **full rotations** (1.0 = 360°, so 0.5 = 180°). Text height should always be **1.000**.

Every province must have position data defined, or units and cities become invisible. Use the **Nudge tool** (**-nudge** launch parameter in EU4, Nudge button in HOI4 debug mode) for visual editing.

Administrative hierarchies

EU4 uses a three-tier system: **area.txt** → **region.txt** → **superregion.txt**. Every province must belong to an area (missing = CTD on interaction). Areas group 3–5 provinces; regions group areas; superregions group regions. Sea regions exist but do not belong to superregions.

HOI4 uses: States (**history/states/*.txt**) → Strategic Regions (**map/strategicregions/*.txt**). States contain province lists, ownership, victory points, and building slots. Strategic regions contain provinces (not states), weather data, and naval terrain for naval regions. Every province must be in exactly one strategic region.

CK3 uses nested titles: Barony (b_) → County (c_) → Duchy (d_) → Kingdom (k_) → Empire (e_), all defined in a single nested structure in **common/landed_titles/00_landed_titles.txt**:

```
e_britannia = {
    k_england = {
        d_york = {
            c_york = {
                b_york = { province = 1623 }
                b_selby = { province = 1624 }
            }
        }
    }
}
```

Vic3 uses: Province → State Region → Strategic Region. State regions are defined in `map_data/state_regions/` with hub assignments:

```
STATE_OHIO = {
    id = 245
    provinces = { x1A2B3C x4D5E6F x7A8B9C }
    city = "x1A2B3C"
    farm = "x4D5E6F"
    mine = "x7A8B9C"
    wood = "x123456"
    port = "xABCDEF"           # coastal states only
    naval_exit_id = 300        # adjacent sea state ID
    impassable = { }
    prime_land = { }
    traits = { "state_trait_name" }
}
```

Every Vic3 land state needs **4 hub types** (city, farm, mine, wood) requiring a minimum of 4 provinces, or **5 for coastal states** (adding port). The hub system is unique to Vic3 — hubs define connectivity between states and determine where settlement graphics appear. **Type 0 hubs** connect adjacent land states; **Type 1 hubs** handle non-adjacent connections (naval, cross-water). Never connect Type 0 to Type 1 — this causes crashes.

Province climate, continents, and terrain definitions

continent.txt (EU4/HOI4): Every land province must appear in exactly one continent block. Missing a province causes a **CTD when clicking that province** in EU4. Sea provinces have continent = 0 in HOI4's definition.csv.

climate.txt (EU4):

```
tropical = { 2100 2101 ... }
arid = { 3050 3051 ... }
mild_winter = { ... }
normal_winter = { ... }
severe_winter = { ... }
impassable = { ... } # wasteland provinces
```

Provinces not listed in any category default to **temperate** (the best climate). Impassable provinces are defined here, not in default.map (unlike CK3).

Phase 5: Modern tooling — the Jomini map editor

Accessing the editor

CK3 and Vic3 share the same map editor framework. There are two methods to launch it:

- **Launch option (recommended):** Right-click the game in Steam → Properties → General → Launch Options → type `-mapeditor`. The editor opens alongside the game on next launch.
- **Console command:** Enable debug mode, open the console with `~`, type `map_editor`. This method may cause terrain visibility issues in Vic3; the launch option is preferred for total conversions.

Additional recommended launch options: `-filewatcher` (auto-reloads modified files) and `debug_mode` (enables error logging and console).

Editor capabilities and limitations

The Jomini map editor provides five primary toolsets:

Heightmap tools: Import external PNG heightmaps via the import button. Use the selector tool to choose the import region, click Accept. The **Resolution tool** (shortcut: `5`) adjusts heightmap detail per tile — use `Numpad +` to increase resolution for tiles that show holes or artifacts. After any heightmap modification, you **must repack** (generates `packed_heightmap.png` and `indirection_heightmap.png`). Without repacking, rivers, borders, and title positions will not conform to the new terrain.

Terrain/material painting: Paint terrain textures (grass, sand, snow, rock) directly onto the map surface using adjustable brushes. Each terrain type corresponds to a mask PNG in `gfx/map/terrain/`. To completely clear a terrain type, open its `_mask.png` file externally and paint it solid black.

Object painting: Place 3D map objects (trees, shrubs, rocks, buildings) using brush tools. Objects are controlled by mask files in `content_source/map_objects/masks/`. The **auto-nudge** tool (dice button) randomizes object positions within province boundaries and auto-generates locator files.

Province and state assignment (Vic3): The editor can paint new provinces onto `provinces.png` and assign provinces to state regions. Saved output appears in `Documents/Paradox Interactive/Victoria 3/generated/` and must be manually copied to the mod folder.

Spline network editing (Vic3 only): The strip tool (shortcut: `9`) creates road, railroad, and naval connection splines between state hubs. Railroads define where rail graphics appear when built in-game. Object tools (shortcut: `@`) should have "auto-refresh" and "draw city spheres" enabled for visual feedback.

What the editor cannot do:

- Edit `rivers.png` — must use GIMP/Photoshop externally
- Edit `definition.csv` — must use a text editor
- Assign province terrain types in CK3 — set in `common/province_terrain/`
- Auto-generate terrain in Vic3 — use `terrain.regenerate` console command instead
- Edit history files, population data, or building assignments

Vic3-specific console commands

- `map_editor` — opens the map editor from within the game
- `terrain.regenerate` — auto-generates `province_terrains.txt` from geography trigger scripts. Prerequisites: geography triggers must be configured in `common/scripted_triggers/00_geography_triggers.txt` and terrain rules in `common/terrain/01.terrain.txt`. Output goes to the `generated/` folder.

Community tools for the complete pipeline

The modding community has developed a robust ecosystem of open-source tools that fill gaps in the official tooling:

Province generation and editing:

- **RandomParadox** (github.com/panik4/RandomParadox) — C++ procedural worldmap generator supporting HOI4 (full), Vic3 and EU4 (experimental). Generates heightmaps via FastNoiseLite, province maps, terrain, climate zones, countries, and complete mod folder structures. The most comprehensive automated pipeline available.

- **HOI4PE / hoi4_province_editor** (github.com/ScottyThePilot/hoi4_province_editor) — Rust-based graphical province editor for HOI4. Edits `provinces.bmp` and `definition.csv` simultaneously with terrain/biome map modes, coastal detection, and adjacency creation.
- **Clausewitz-Color-Factory** (github.com/DerekWarter/Clausewitz-Color-Factory) — CLI tool generating 1-50,000 unique province RGB colors with configurable minimum contrast and brightness sorting.
- **Provincial** (github.com/ThomasSlade/Provincial) — Python scripts for HOI4 province validation: `validateprovinces.py` checks for X-crossing errors, `assignprovinces.py` assigns provinces to states by comparing province maps to boundary images.

Parsing and conversion:

- **rakaly/jomini** (github.com/rakaly/jomini) — High-performance Rust parser for all Paradox save and game files, handling both plaintext and binary formats.
- **Clausewitz Scenario Editor** (github.com/mmyers/eug, also on SourceForge) — Java-based editor for EU4 and CK3 scenarios, includes a positions editor.
- **MapDegrader** (github.com/ParadoxGameConverters/MapDegrader) — Converts CK3 barony-level province maps to county-level maps for game converter pipelines.

Validation and debugging:

- **Clausewitz Map Checker** (Paradox Forums) — .NET tool that opens `definition.csv` + province map and highlights unknown/stray pixels.
- **Error logs** are written to `Documents/Paradox Interactive/[Game]/logs/` — check `error.log`, `setup.log`, and `exceptions.log`.
- HOI4's **Nudger** (accessible via debug mode) provides visual province inspection, strategic region editing, supply node placement, building positioning, and unit stack arrangement across multiple tabs.

The step-by-step pipeline from blank canvas to bootable map

This sequence represents the optimal order of operations distilled from community experience across all four games:

1. **Create the heightmap** at target resolution. Clausewitz: 8-bit greyscale BMP (sea level at 94). Jomini: 16-bit greyscale PNG. Ensure smooth gradients. For CK3/Vic3, import into the map editor and repack.

2. **Paint the province map** using unique solid RGB colors with zero anti-aliasing. Every pixel must be exactly one province color. Generate unique colors with Clausewitz-Color-Factory or use RandomParadox for automated generation.
3. **Create definition.csv** (EU4/HOI4/CK3 only). Map each RGB color to a province ID and name. HOI4 can auto-generate a fixed version on first launch. Vic3 skips this step entirely.
4. **Configure default.map** — declare dimensions (Clausewitz), max_provinces, sea province lists, file references, and province categories (Jomini).
5. **Create terrain** — Clausewitz: paint `terrain.bmp` (indexed) and configure `terrain.txt`. CK3: set terrain per province in `common/province_terrain/`. Vic3: configure geography triggers and run `terrain.regenerate`.
6. **Draw rivers** in `rivers.bmp`/`rivers.png` using the exact palette colors. One green source per system, 1-pixel width, orthogonal adjacency only. Preserve the vanilla color palette.
7. **Generate the normal map** from the heightmap (Clausewitz only). Must be exactly half resolution for EU4 colormaps.
8. **Create colormaps/DDS files** — seasonal overlays (EU4), colormap with emissive mask (HOI4), or terrain masks (CK3/Vic3).
9. **Build administrative structures** — areas/regions/superregions (EU4), states/strategic regions (HOI4), landed titles hierarchy (CK3), or state regions/strategic regions with hub assignments (Vic3).
10. **Set up continent.txt** (EU4/HOI4) — assign every land province. Missing entries cause CTDs.
11. **Configure positions** — `positions.txt` (EU4/HOI4) or locator files (CK3/Vic3). Use the Nudge tool or map editor auto-nudge for placement.
12. **Create adjacencies.csv** with the mandatory `-1;-1;;-1;-1;-1;-1;-1` terminator line.
13. **Validate** — launch with debug mode, check error logs, use the Clausewitz Map Checker, fix stray pixels, verify every province has continent/area/state assignments.

Conclusion

The Paradox map pipeline has maintained remarkable architectural consistency across two engine generations. The RGB-province paradigm — where every pixel of a specific color belongs to a specific province — has survived from EU3 through Vic3 essentially unchanged. What has evolved is the precision and tooling around this system. Jomini's 16-bit heightmaps offer **256× more elevation resolution** than Clausewitz's 8-bit version. The integrated map editor eliminates the need for blind bitmap editing of terrain and objects. Vic3's elimination of `definition.csv` in favor of hex-color-as-ID is the most radical simplification yet, removing an entire class of color-mismatch bugs.

The most critical insight for anyone building a map from scratch: **the province bitmap is sacred ground**. A single anti-aliased pixel, a single duplicate color, a single province missing from `continent.txt` — any of these will crash the game silently or spectacularly. Every other layer in the pipeline is relatively forgiving by comparison. Start with a clean province map, validate it obsessively, and build outward from there.